



RAG with ChromaDB

Building a Local Knowledge Base for Grounded Retrieval Agents

A practical guide for software engineers and ML practitioners who want to move beyond prompt-only agents and build systems that answer from evidence — not inference.

Before We Start

This session builds quickly. Before diving in, check your footing across four areas. Gaps are fine — the goal is to establish the mental model before touching any implementation.

1

Python and Data

Can you read structured records like product entries or FAQs and imagine turning them into retrievable text units? Are you comfortable reasoning about splitting content into smaller chunks?

2

LLM Basics

Do you understand the difference between a model generating a fluent answer and a system grounding that answer in external evidence? Are you comfortable with the idea that a good agent should sometimes refuse to answer?

3

Search Intuition

Have you seen how a question can be relevant to a document even when exact keywords don't match? Do you have an intuition for why semantic similarity outperforms literal keyword overlap?

4

Agent Mindset

Can you picture an agent that first retrieves evidence, then answers from it — instead of relying on memory alone? Are you ready to judge answer quality by groundedness, not just fluency?



Why Does an Agent Need Retrieval?

What Goes Wrong Without It

- The agent relies on model memory — which is incomplete, stale, and unverified for your specific catalog
- Product specs, warranty rules, and FAQ policies change; model knowledge does not
- Confident-sounding answers can still be factually wrong when not grounded in current data

What Retrieval Adds

- Access to the **current knowledge base** at question time — not snapshot training data
- Answers drawn from actual product and FAQ evidence, not background inference
- A clear boundary between what the agent *retrieved* and what it is *allowed to say*

 Retrieval is not an enhancement — it is the mechanism that makes factual accuracy possible at all.

What Problem Does RAG Solve?

Retrieval-Augmented Generation restructures the agent's relationship with knowledge — shifting from memory-based inference to evidence-driven answers.

The Prompt-Only Limitation

- Strong at generating coherent language
- Not tied to the latest catalog, internal FAQ, or store-specific policy
- No distinction between "I retrieved this" and "I inferred this"

How RAG Changes the Flow

- Retrieval step happens **before** generation — evidence first, answer second
- System finds relevant passages, then generates an answer directly from them
- The knowledge base drives the answer, not model memory

What Becomes Possible

- Answers aligned with stored, verifiable facts
- Graceful fallback when the answer is not supported by the knowledge base
- A foundation that scales as the knowledge base grows — without retraining

What Are Embeddings?

The Core Idea

An embedding is a numeric vector that encodes the **semantic meaning** of a piece of text. Similar meanings end up geometrically close together in vector space — regardless of surface wording.

Consider: *"Battery backup," "power duration,"* and *"how long does it last?"* can all resolve to the same stored passage because they share intent, not literal terms.

What Drives Quality

The embedding model determines how well meaning is captured. Stronger models produce richer, more nuanced representations. Even a lightweight local model is sufficient for most catalog and FAQ retrieval tasks — you don't need a heavyweight model to start.

❏ Why this matters for retrieval

Users rarely ask questions in the exact wording of the source document. Embedding-based retrieval bridges the gap between **natural language queries** and **stored content** — matching by intent and meaning, not literal phrase overlap.

Input Text

Any string — query, document chunk, or FAQ answer

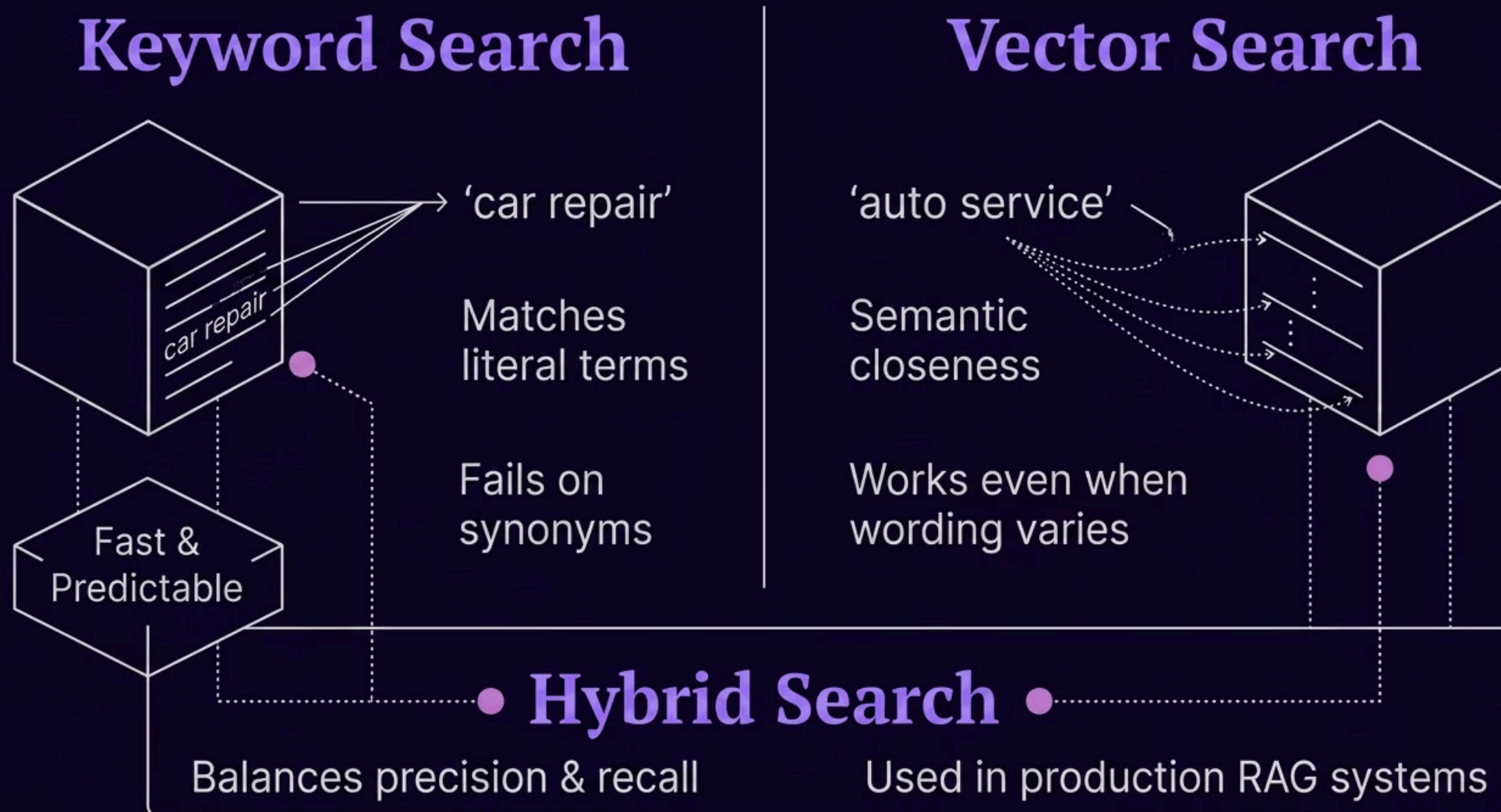
Dense Vector

Fixed-length array of floats encoding semantic content

Vector Space

Similar meanings cluster together; distance = semantic difference

How Is Vector Search Different?



Keyword search is fast and predictable but collapses under paraphrasing. Vector search is meaning-aware and flexible but requires an embedding step. Most production RAG systems combine both — using keyword lookup for structured fields and vector search for open-ended, conversational queries where intent varies but wording does not need to match exactly.

What Does ChromaDB Do?



Local Vector Store

Acts as the local vector database that stores embedded document chunks. Accepts queries as embeddings and returns the closest matching passages — entirely locally, with no cloud account or external API required.



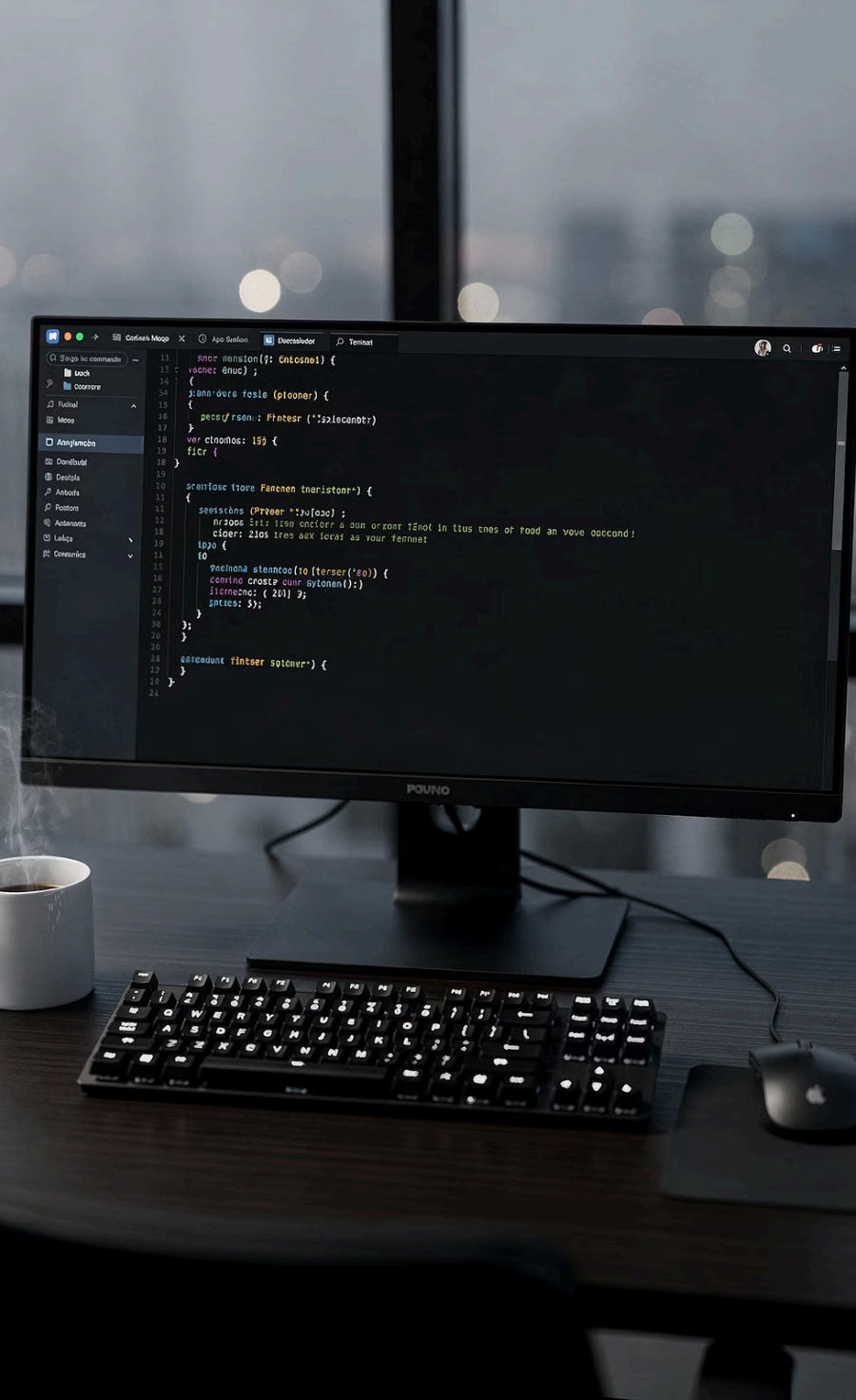
Easy to Wire In

Lightweight and straightforward to integrate into a Python-based agent. Provides a real retrieval layer without the complexity of a managed vector backend — same conceptual pattern that transfers directly to cloud stores later.



Inspectable Source of Truth

Gives the agent a retrievable evidence layer that is separate from the model. Answers come from stored facts rather than generative inference, and the source of each response remains clear and auditable.



How Do Documents Become Retrievable?

What Should Go Into the Knowledge Base?

- Product catalog entries — specifications, pricing, compatibility, availability
- FAQ content — warranty terms, return policy, support procedures
- Any content the agent must answer factually and without guessing

⚠ Retrieval is only as good as the content you put in. Missing or stale documents silently degrade answer quality.

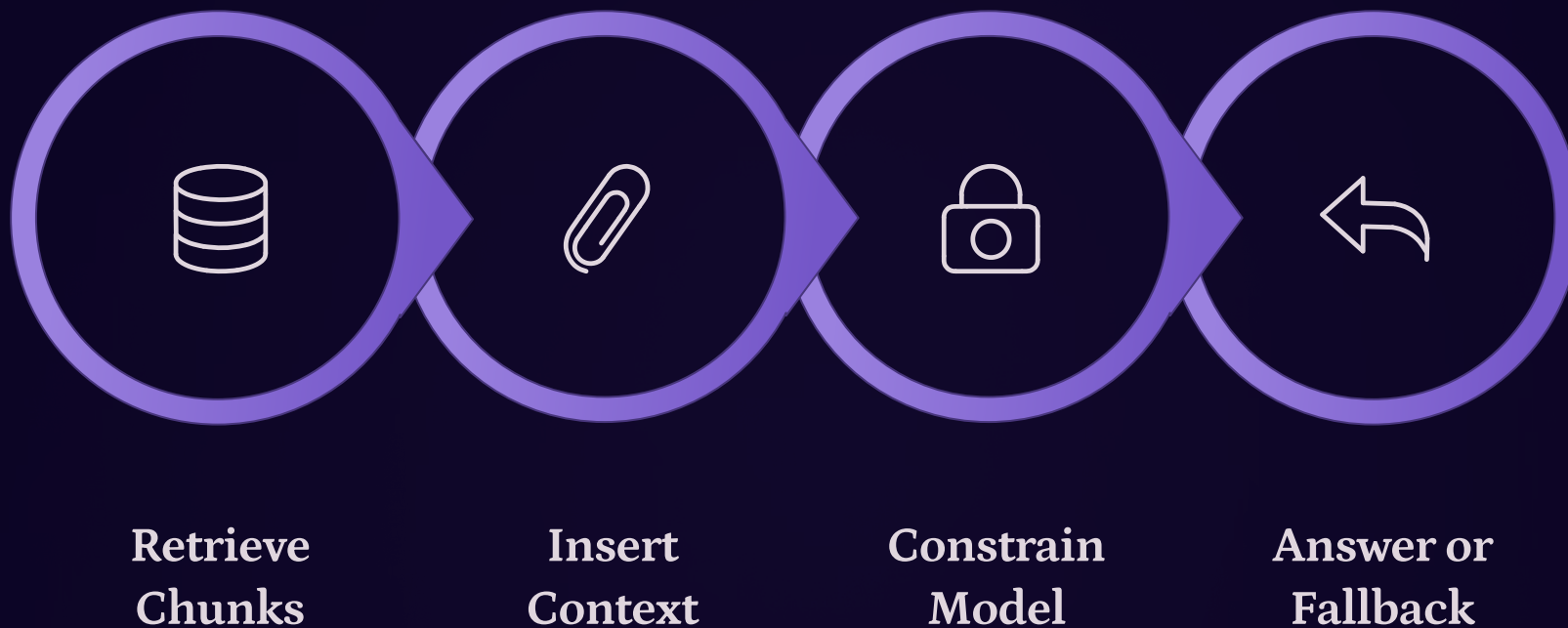
Why Chunking Matters

Storing everything as one large document makes retrieval noisy. The system cannot distinguish what is relevant from what is background.

- **Too large:** Irrelevant context appears alongside useful content
- **Too small:** Evidence is fragmented; answers become incomplete
- **Right size:** Each chunk is a self-contained, meaningful unit — headers and category labels help the model use it correctly

Chunk size is a **tuning decision**, not a one-time setup choice.

How Does Context Injection Work?



After retrieval, the top-ranked chunks are inserted into the prompt as context. The model is explicitly instructed to answer using **only that context** — not background training knowledge. Both the query and the retrieved evidence are visible to the model at generation time. Without a clear instruction, the model can still drift beyond what was retrieved, silently mixing evidence with inference. The prompt must enforce the rule: if the context does not support the answer, the model must say so rather than fill the gap.

Why Must the Agent Answer Only From Evidence?

The Risk of Weak Grounding

A fluent model can combine weak retrieval with confident language. The result is a believable but false answer. Users have no way to detect the difference without seeing the source — and most never will.

- Hallucinated product specifications
- Invented warranty terms or return policies
- Confident errors that erode trust over time

The Mindset Shift

Evaluating answer quality requires two independent checks — not one.

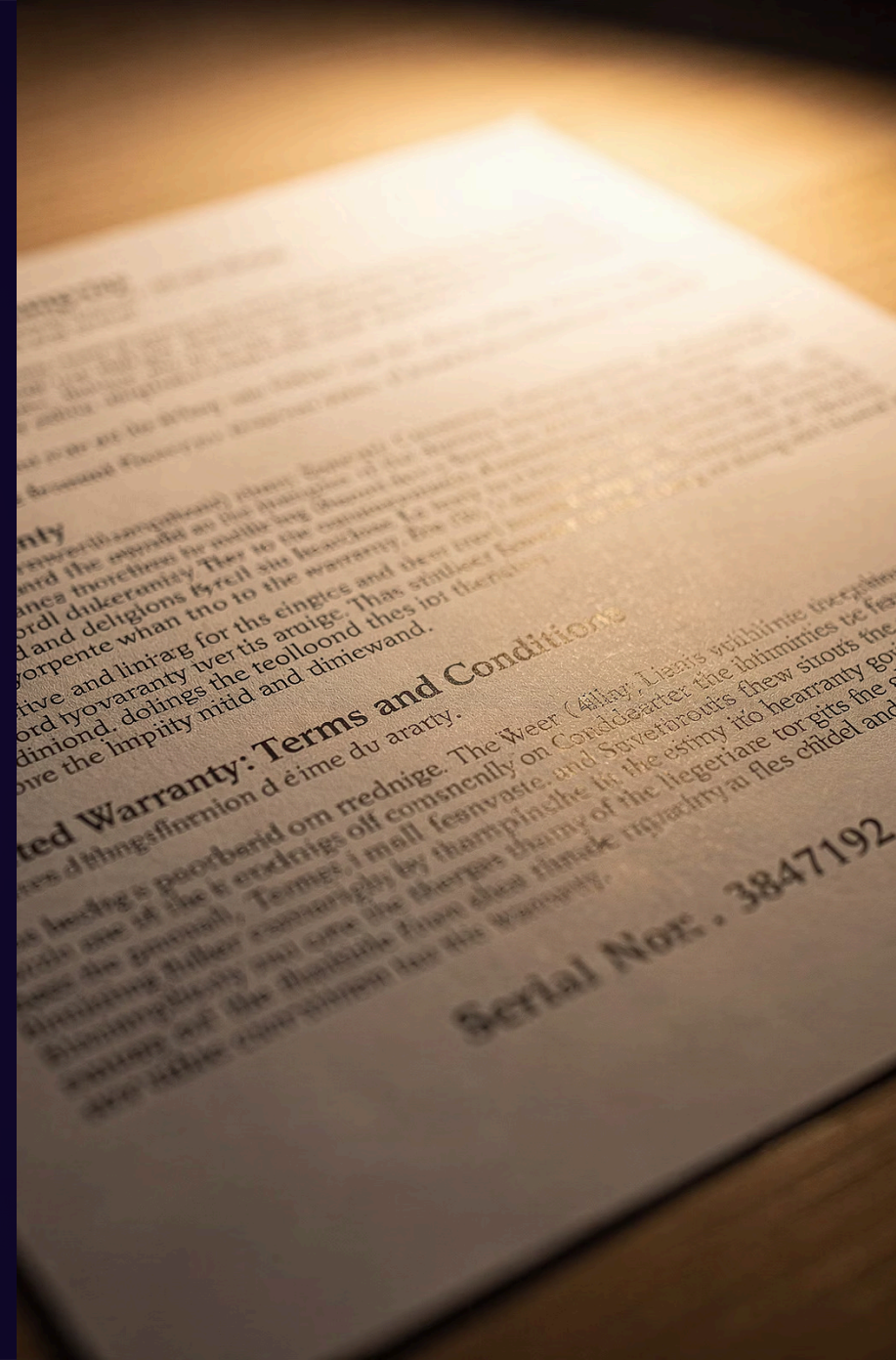
Is it helpful?

Does the answer address what the user asked?

Is it grounded?

Is every claim supported by what was actually retrieved?

Correctness and groundedness are both first-class requirements. A helpful but ungrounded answer is not a success.



What Is a Hallucination Guard?

A hallucination guard is not a single feature — it is a **design discipline** applied at every step of the RAG pipeline to prevent the model from inventing facts the knowledge base does not support.

What the Term Means

The combination of retrieval checks, answer constraints, and fallback behavior. Keeps the model within the boundaries of what was actually retrieved — not what it could plausibly generate.

What It Looks Like in Practice

Model instructed to answer only from retrieved evidence. An explicit fallback phrase when evidence is weak or irrelevant. A retrieval score or relevance check before passing context to the model at all.

Why It Is Essential

Without it, a retrieval system can still fail politely. Wrapping low-quality evidence in high-confidence language is **worse than saying "I don't know."** The hallucination guard is what separates a demo from a trustworthy production system.

What Should Happen When Retrieval Is Weak?

Wrong Chunk Retrieved

When an irrelevant or loosely related chunk is returned, the answer becomes plausible but factually wrong. This failure mode is **harder to detect** than a complete retrieval miss — a confident wrong answer is more dangerous than no answer at all.

- ⊗ Retrieval quality and grounding rules must be tested together. One without the other gives false confidence.

No Relevant Chunk Retrieved

The system should return a graceful, honest fallback — for example:

"I don't have that information in the current knowledge base."

The fallback must not be fabricated content dressed up as a polite response. A careful **"I don't know"** is more trustworthy than a confident invention. Users and support teams can act on a fallback — they cannot act on a hallucinated spec.

How Do You Test a RAG System?

Testing a RAG system requires deliberate coverage across four query types — including the failure cases that matter most in production.



Correct Matches

Questions the knowledge base directly supports. Verify the answer reflects the retrieved source — no unsupported additions or generative drift. The source document must be traceable.



Near Matches

Questions that partially overlap with stored content. Watch for the model silently filling gaps with inference. The answer should reflect only what the chunk actually supports.



Wrong Matches

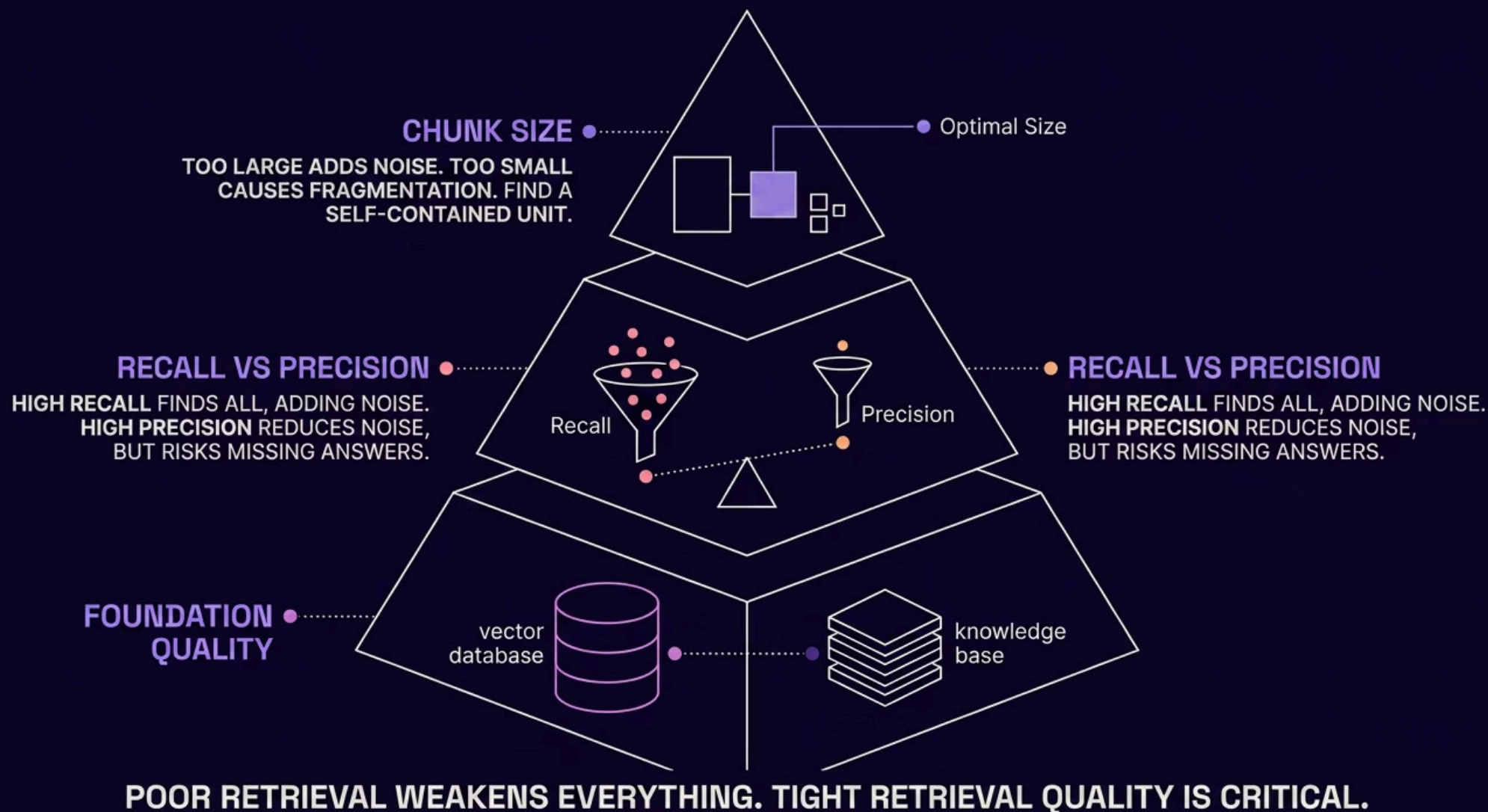
Questions that sound similar to a stored passage but should not return a confident answer. The system must not surface a plausible-but-wrong chunk as authoritative evidence.



Unanswerable Queries

Queries with no relevant content in the knowledge base. These must trigger fallback — not invention. Test these cases explicitly. They are the ones that matter most in production.

What Trade-Offs Matter Most?



Every later capability built on grounded answers depends directly on retrieval quality. Chunk size determines whether evidence arrives intact. The recall-precision balance determines whether the right chunk is even a candidate. Both are tuning decisions that need revisiting as the knowledge base and query patterns evolve — they are not one-time configuration steps.

The Minimal Mental Model

The RAG Flow in Plain Terms

01

User submits a query

02

Query is embedded into a vector

03

Vector store returns the closest matching chunks

04

Chunks are injected into the prompt as context

05

Model generates an answer from that context

06

Fallback fires if context is weak or absent

Why This Is a Foundational Shift

The agent no longer answers from prompt and model memory alone. Every answer has a **retrievable evidence trail**. Knowledge base updates automatically improve answers — without retraining the model.

This moves the system from *generative assistant* toward *knowledge-grounded agent* — more durable, more auditable, and more trustworthy for real support use cases.

✓ The same pattern carries forward into every more advanced architecture that follows.

What to Explore Next

You've built the mental model for local, grounded retrieval. These open questions point toward the next layer of complexity — keep them close as you move forward.

→ **Scaling Beyond Local Storage**

How does retrieval change when a local vector store is no longer sufficient for the size or update frequency of the knowledge base?

→ **Moving to a Managed Backend**

What trade-offs appear when the same grounding pattern moves from local storage to a managed cloud vector store?

→ **Surfacing Evidence in the UI**

How should a system expose retrieved sources and evidence in the user interface without overwhelming the user or undermining trust?

→ **Complex Agent Decisions**

What happens when grounded retrieval is combined with model routing, multi-step fallbacks, and more sophisticated agent orchestration?

→ **Evaluating Retrieval Over Time**

How do you detect whether retrieval quality is improving, drifting, or silently degrading — before it affects user-facing answers?